# SimEvents Manufacturing Cell — First Run Results

Two-stage manufacturing cell with inspection and rework loop | MATLAB SimEvents

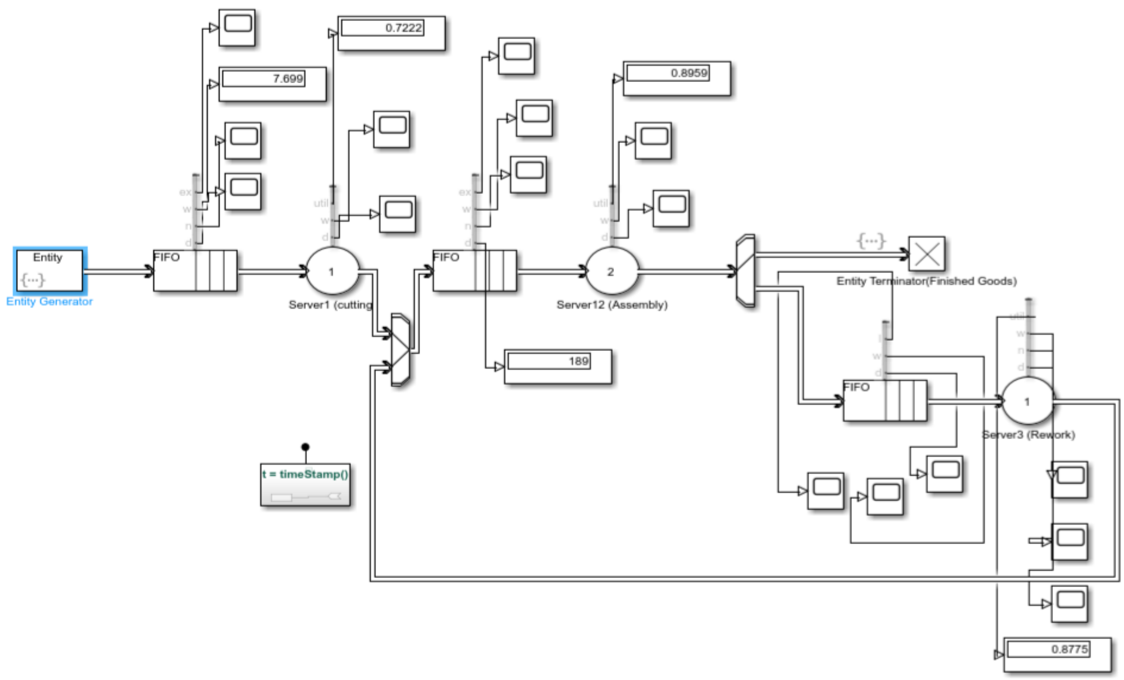
## Project summary

This is a discrete-event simulation of a small manufacturing cell built in MATLAB SimEvents. Jobs arrive randomly, pass through a cutting station and a two-operator assembly station, and are then inspected. Passed jobs exit as finished goods; failed jobs are routed through a rework loop before re-entering the assembly buffer. The model demonstrates entity generation, queuing, resource-constrained servicing, probabilistic routing, feedback loops, and statistics collection.

## Run configuration (initial test run)

| Parameter | Value |
|---|---|
| Job arrival (inter-arrival time) | Exponential, mean = 5 min |
| Cutting station service time | Exponential, mean = 4 min, capacity 1 |
| Assembly station service time | Exponential, mean = 6 min, capacity 2 (2 operators) |
| Rework station service time | Exponential, mean = 8 min, capacity 1 |
| Inspection pass / fail split | 85% pass / 15% fail (rework) |
| Simulation stop time | 100 minutes (initial smoke-test run) |

## Model diagram (as run)




Simulink/SimEvents block diagram: Entity Generator → Cutting station → Entity Input Switch (merges rework feedback) → Assembly station (2 operators) → Entity Output Switch (85/15 split) → Finished Goods / Rework loop. Display and Scope blocks attached at each station show live statistics.

## Results from this run

| Metric | Value | Interpretation |
|---|---|---|
| Cutting station utilization | 72.2% | Busy most of the time, some slack remains |
| Assembly station utilization | 89.6% | Near-saturated — likely bottleneck |
| Rework station utilization | 87.8% | Heavily loaded despite only ~15% of jobs routing here |
| Entities through assembly queue | 189 | Confirms steady flow over the run |
| Average cycle time (mean(cycleTimes)) | 28.31 min | Average time a job spends in the system, generation to finished goods |

## Observation

Assembly (89.6%) and rework (87.8%) are both running near saturation, while cutting (72.2%) has meaningful slack. In queuing theory, utilization above roughly 80–85% is where queue lengths and wait times begin growing sharply rather than linearly — so assembly is close to becoming a hard bottleneck in this cell. This is a direct, testable insight the model produced, not an assumption built into it.

## Note on this run

This run used a short stop time (100 minutes, ~15–20 entities generated) as an initial smoke test to confirm the model runs correctly end to end and produces plausible numbers. A longer run (e.g. 1000 minutes) is planned next to let the statistics reach steady state before treating these numbers as final — short runs can understate congestion because the system hasn't had time to build up queues from its initially empty state.

## MATLAB actions used in the model

The following MATLAB expressions were entered directly into block dialogs (Service time action, Intergeneration time action, and Event actions fields) to drive randomness, routing, and timestamping in the model.

| Block / field | MATLAB action |
|---|---|
| **Entity Generator<br>Intergeneration time action** | `dt = -5*log(rand);` |
| **Entity Generator<br>Event actions → Generate** | `entity.Attribute1 = (rand() > 0.85) + 1;`<br>`entity.genTime = timeStamp();` |
| **Server1 (Cutting)<br>Service time action** | `dt = -4*log(rand);` |
| **Server2 (Assembly)<br>Service time action** | `dt = -6*log(rand);` |
| **Server3 (Rework)<br>Service time action** | `dt = -8*log(rand);` |
| **Entity Terminator<br>Event actions → Entry** | `persistent cycleTimes`<br>`if isempty(cycleTimes)`<br>`  cycleTimes = [];`<br>`end`<br>`ct = timeStamp() - entity.genTime;`<br>`cycleTimes(end+1) = ct;`<br>`assignin('base', 'cycleTimes', cycleTimes);` |
| **Simulink Function block<br>(f() subsystem)** | `function t = timeStamp()`<br>`  % Digital Clock (sample time -1)`<br>`  % wired straight to output t`<br>`end` |

## Learnings

- Entity generation with stochastic (exponential) inter-arrival times
- FIFO queuing with finite capacity
- Resource-constrained service using server capacity (parallel operators)
- Probabilistic entity routing (Entity Output Switch, attribute-driven)
- Feedback loops merged via Entity Input Switch
- Built-in statistics collection (utilization, wait time, queue length, throughput)
- Custom end-to-end cycle time tracking via Simulink Function + event actions
- Run-length / steady-state considerations in discrete-event simulation